

IT Workshop II (Cloud Computing): Cloud Application Design

September 9, 2026

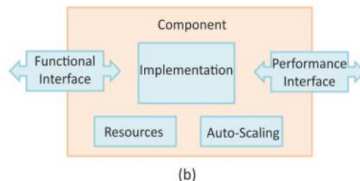
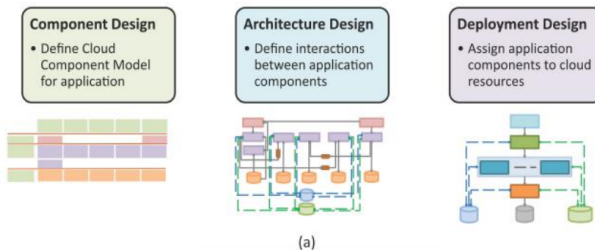
Cloud Component Model

- Cloud Component Model is an application design methodology that provides a flexible way of creating cloud applications in a rapid, convenient and platform independent manner.
- CCM is an architectural approach for cloud applications that is not tied to any specific programming language or cloud platform.
- Cloud applications designed with CCM approach can have innovative hybrid deployments in which different components of an application can be deployed on cloud infrastructure and platforms of different cloud vendors.
- Applications designed using CCM have better portability and interoperability.

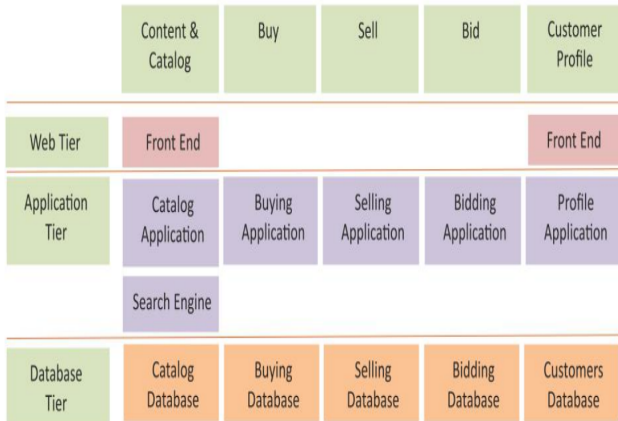
CCM Application Design Methodology

- CCM based applications have better scalability by decoupling application components and providing asynchronous communication mechanisms.
- CCM approach for application design involves:
 - Component Design
 - Architecture Design
 - Deployment Design

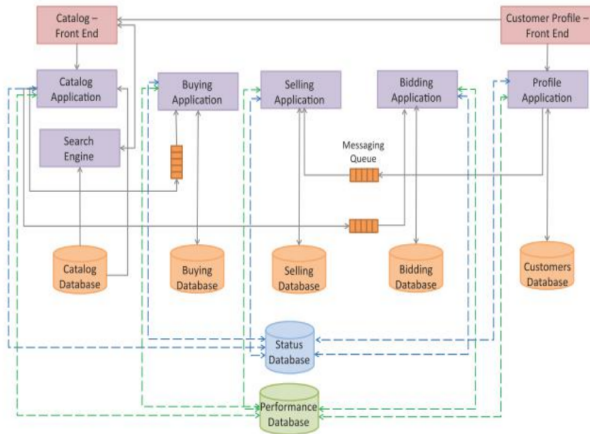
CCM Application Design Methodology



CCM Component Design

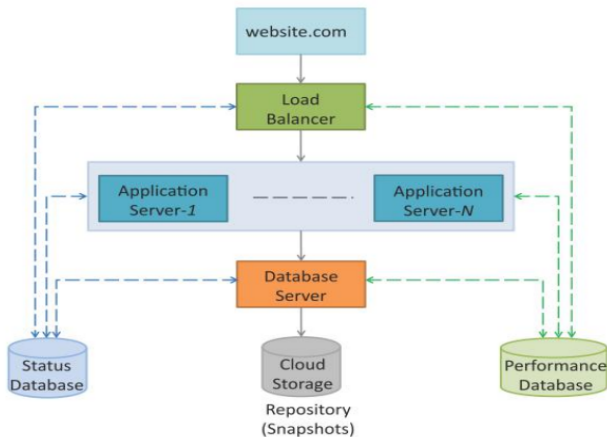


CCM Architecture Design



- **Loose Coupling:** Components in the Cloud Component Model are loosely coupled.
- **Asynchronous Communication:** By allowing asynchronous communication between components, it is possible to add capacity by adding additional servers when the application load increases. Asynchronous communication is made possible by using messaging queues.
- **Stateless Design:** Components in the Cloud Component Model are stateless. By storing session state outside of the component (e.g. in a database), stateless component design enables distribution and horizontal scaling.

CCM Deployment Design



CCM Deployment Design

- In Deployment Design step, application components are mapped to specific cloud resources such as web servers, application servers, database servers, etc.
- Since the application components are designed to be loosely coupled and stateless with asynchronous communication, components can be deployed independently of each other.
- This approach makes it easy to migrate application components from one cloud to the other.
- With this flexibility in application design and deployment, the application developers can ensure that the applications meet the performance and cost requirements with changing contexts.

Differences

	SOA	CCM
End points	SOA services have small and well-defined set of endpoints through which many types of data can pass.	CCM components have very large number of endpoints. There is an endpoint for each resource in a component, identified by a URI.
Messaging	SOA uses a messaging layer above HTTP by using SOAP which provide prohibitive constraints to developers.	CCM components use HTTP and REST for messaging.
Security	Uses WS-Security , SAML and other standards for security	CCM components use HTTPS for security.
Interfacing	SOA uses XML for interfacing.	CCM allows resources in components represent different formats for interfacing (HTML, XML, JSON, etc.).
Consumption	Consuming traditional SOA services in a browser is cumbersome.	CCM components and the underlying component resources are exposed as XML, JSON (and other formats) over HTTP or REST, thus easy to consume in the browser.

Model View Controller

- **Model:** Model manages the data and the behavior of the applications. Model processes events sent by the controller. Model responds to the requests for information about its state (from the view) and responds to the instructions to change state (from controller).
- **View:** View prepares the interface which is shown to the user. Users interact with the application through views. Views present the information that the model or controller tell the view to present to the user and also handle user requests and sends them to the controller.
- **Controller:** Controller glues the model to the view. Controller processes user requests and updates the model when the user manipulates the view. Controller also updates the view when the model changes.

Model View Controller

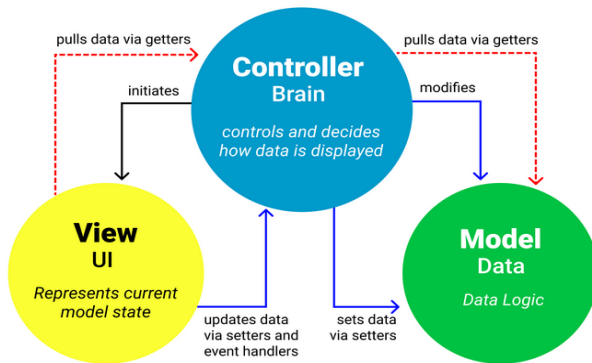


Figure: Model View Controller (Source: Internet)

The End